

SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO2 AT1 – Capstone Infrastructure Sizing Workbook

Guided calibration workbook for VM sizing, virtualization decisions, snapshot planning and VM-container comparison

Assessment Metadata

Field	Details
Course Code	CSA15
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT1 - Virtualization Scenario Problem-Solving
Submission Type	Individual digital workbook based on the same capstone title used in CO1
Faculty	Dr. C. Rajesh Babu
Employee ID	SSETSCS-415
Submission Mode	Completed workbook and evidence repository link through Google Classroom

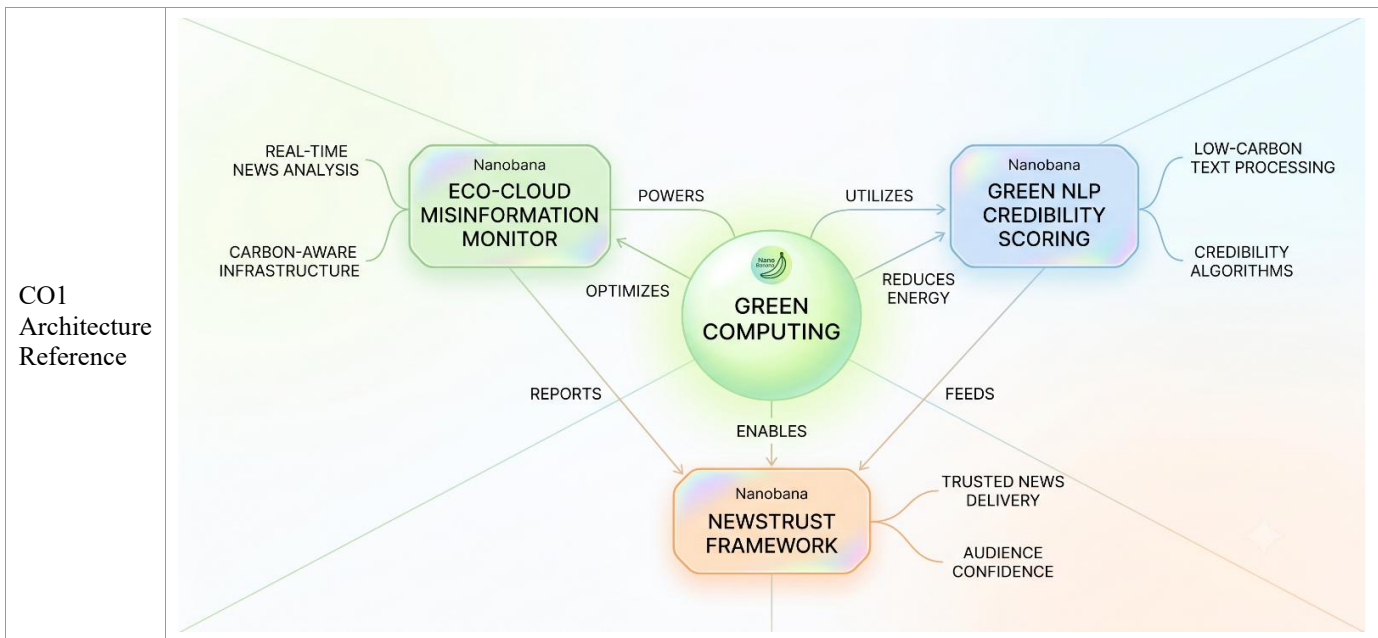
How This Workbook Works

Read each guidance block and immediately fill the related response table. Use the same capstone title selected for CO1. Make reasonable assumptions when exact values are unknown, and state each assumption clearly. The goal is not to guess a perfect industry configuration; the goal is to reason technically and justify the first infrastructure plan. The final decisions from this workbook will be used again in CO2 AT2 and CO2 AT3.

Technical Tip: A good infrastructure estimate is transparent. If a number is assumed, write why it is assumed. If a number is calculated, show the formula. If a service is selected, justify why it fits the workload.

Student and Capstone Details

Field	Student Entry
Student Name	K Dheepak
Register Number	192411108
Class / Slot	Slot C
Capstone Title	News Trust: Cloud Misinformation Monitor Using NLP Credibility Scoring
Capstone Product Type	Web app



Activity 1: Bring Forward CO1 Capstone Context

Start with the cloud product idea already used in CO1. The infrastructure plan must support that same application, not a new unrelated example.

Capstone Element	Student Entry
Primary users	General news readers, independent fact-checkers, journalists, and content moderation teams at partner platforms
User roles	Admin, Fact-Checker/Analyst, Registered Reader, Guest (read-only)
Main modules	Article ingestion & scraping, NLP credibility scoring engine, source reputation tracker, analyst dashboard & alerts, user reporting/flagging module
Cloud services identified in CO1	Compute VM for API, managed relational database, object storage for article snapshots, ML inference service, message queue for the ingestion pipeline
API/backend need	Yes - REST API for article ingestion, credibility score retrieval, dashboard data and user reports
Database need	Yes - relational database for users, articles, sources and scores, plus a lightweight search/index layer for fast lookup
File upload/storage need	Yes - stored HTML/screenshot snapshots of scraped articles, exported PDF fact-check reports and source logos
Analytics/AI/Python module	Yes - Python NLP pipeline (transformer-based text classifier) for credibility scoring, bias detection and claim similarity matching

Activity 2: Workload Classification

Classify the capstone workload before selecting a VM. A form-based SaaS app and an AI analytics app should not receive the same infrastructure plan.

Workload Type	Typical Signals	Starting VM Guidance
Light web / form app	Login, CRUD, simple reports, low file upload	1-2 vCPU, 2-4 GB RAM
API backend	Authentication, REST APIs, moderate concurrent requests	2 vCPU, 4-8 GB RAM
Database-heavy app	Search, many records, frequent writes	2-4 vCPU, 8-16 GB RAM
File-heavy app	Images, PDFs, uploads, downloads	2 vCPU, 4-8 GB RAM + separate object storage
Analytics app	Dashboards, aggregation, batch reports	4 vCPU, 16 GB RAM
AI/Python module	Prediction, NLP, image/text processing	4-8 vCPU, 16-32 GB RAM; GPU optional
High traffic app	Large user base and peak demand	Multiple VMs or autoscaling group

Technical Tip: Start small for prototype, but do not hide growth. A valid answer can say: prototype uses one VM, pilot separates database, production uses load balancing and managed services.

Student Decision: My capstone workload type and reason

Workload type: AI/Python module, layered with an API backend. NewsTrust's core function is running an NLP transformer model to score every ingested article for credibility, which is CPU/GPU and memory intensive, while the dashboard and reporting features add moderate, steady API traffic. I am starting with a single combined VM for the prototype (1-2 vCPU is not enough here because of the NLP inference load), and I plan to separate the AI module into its own service once traffic grows, consistent with the AI/Python module guidance of 4-8 vCPU, 16-32 GB RAM.

Activity 3: User and Traffic Calibration

Use the following formulas to convert user assumptions into approximate traffic.

Metric	Formula	Example
Concurrent Active Users	Registered Users x Active User % x Peak Factor	$500 \times 10\% \times 2 = 100$
Requests per Minute	Concurrent Active Users x Actions per User per Minute	$100 \times 2 = 200$
Requests per Second	Requests per Minute / 60	$200 / 60 = 3.33 \text{ RPS}$
Daily Requests	Daily Active Users x Avg Actions per Day	$200 \times 30 = 6000$

Input	Value	Reason / Assumption
Registered users expected	2,000	Realistic pilot size for a niche fact-checking web app launched to a university/journalism community
Active user percentage	15%	News-checking tools are used in bursts around breaking news, not daily by everyone
Peak factor	2x	Traffic spikes 2x average during a viral misinformation event
Actions per user per minute	3	A user checks 2-3 articles/scores per minute while browsing the dashboard
Calculated RPS	30 RPS	$2,000 \times 15\% \times 2 = 600$ concurrent users; $600 \times 3 = 1,800$ req/min; $1,800/60 = 30$ RPS
Daily active users	300	15% of 2,000 registered users active on an average day
Daily requests	12,000	$300 \text{ DAU} \times 40 \text{ avg actions/day}$ (article checks + dashboard/report views) = 12,000

Activity 4: CPU Calibration

CPU sizing depends on request rate and processing complexity. Use the score below as a guided estimate.

CPU Driver	Score	
Base web/API application	1	
Authentication and role management	+1	
Moderate API traffic above 5 RPS	+1	
Database-heavy search or reporting	+1 to +2	
Background jobs or notifications	+1	
Analytics or dashboards	+2	
AI/Python inference inside server	+2 to +4	

Total CPU Score	Recommended vCPU
1-2	1-2 vCPU
3-4	2 vCPU
5-6	4 vCPU
7-9	4-8 vCPU or separate analytics/AI service
10+	Scale out with multiple VMs or managed compute

CPU Driver Selected	Score	Reason
Base web/API application	1	Every deployment needs a baseline API server
Authentication and roles	+1	4 distinct roles (Admin, Analyst, Registered Reader, Guest) need auth checks on most routes
API traffic	+1	Calculated load is 30 RPS, above the 5 RPS threshold
Database/search/reporting	+1	Frequent article/source lookups and credibility-score searches
Background jobs	+1	Scheduled scraping jobs and alert notifications run on a queue
Analytics/dashboard	+2	Analyst dashboard aggregates trend and source-reliability statistics
AI/Python module	+3	Transformer-based NLP credibility scoring is the heaviest workload; scored per article in near real time
Total CPU Score	10	Sum of all drivers above
Recommended vCPU	4-8 vCPU for the app tier, with the AI module separated onto its own instance	Score of 10 falls in the 10+ band, which recommends scaling out; splitting AI inference from the API tier keeps each service right-sized instead of over-provisioning one large VM

Activity 5: RAM Calibration

RAM must include operating system, runtime, application process, database/cache and a safety buffer.

RAM Formula: Estimated RAM = OS/runtime + app/API + database/cache + analytics/AI + 25% buffer. If database is managed separately, do not include full database memory in the app VM.

Component	Guidance	Student Estimate
OS + runtime	1-2 GB	2 GB
Web/API service	1-2 GB	2 GB (Node/Django API + Nginx)
Small database	2-4 GB	— (offloaded to managed DB, not counted in app VM)
Medium database/cache	4-8 GB	4 GB (Redis cache for scored articles + session store)
Analytics/reporting	4-8 GB extra	4 GB (dashboard aggregation queries)
AI/Python module	4-16 GB extra	12 GB (transformer model weights + inference batching)
25% buffer	Subtotal x 0.25	Subtotal = 24 GB; buffer = 6 GB
Final RAM	Round up to standard size	30 GB -> rounded up to 32 GB standard instance size

Activity 6: Storage Calibration

Storage must cover application files, database records, uploads, logs and backup/growth buffer.

Storage Item	Formula / Guidance	Student Estimate
OS and application	20-40 GB	30 GB
Database	Avg record KB x records/day x days x 1.3 / 1024 MB	2 KB x 5,000 records/day x 180 days x 1.3 / 1024 = ~2.3 GB
File uploads	Avg file MB x files/day x days x 1.3	0.8 MB x 300 snapshots/day x 90 days x 1.3 = ~27.4 GB (kept in object storage, not VM disk)
Logs	Requests/day x log KB x days / 1024 MB	12,000 req/day x 2 KB x 30 days / 1024 = ~0.7 GB
Backup/growth buffer	Subtotal x 30%	VM-disk subtotal (OS + DB + logs) = 33 GB; buffer = ~10 GB
Final storage	Round up to standard disk size	43 GB -> rounded up to 50 GB VM disk, plus a separate ~30 GB (growing) object storage bucket for article snapshots and reports

Technical Tip: Do not store large uploads permanently inside the VM disk when object storage is available. VM disk is for OS and application. Object storage is better for images, PDFs, media and documents.

Activity 7: Select the VM Plan

Plan	When It Fits	Student Choice
Single VM prototype	Small demo, simple app, low traffic	Selected for the CO2 AT1 prototype demo
Two-tier design	App VM + managed/separate database	Selected for the cloud pilot stage
Three-tier design	Frontend, backend/API and database separated	Target for production growth
Containerized backend	Microservices, portable API, CI/CD	Selected for API and NLP module packaging
Serverless function	Event-based task, notification, small automation	Selected for alert/notification triggers
Managed database/storage	Reliability, backup and scaling requirements	Selected once beyond prototype stage

Final VM Plan: selected plan, vCPU, RAM, storage and reason

Prototype (CO2 AT1): Single VM - 4 vCPU, 16 GB RAM, 50 GB disk, running the API, dashboard and NLP inference together for the demo, since the workload score (10) exceeds what a 1-2 vCPU box can serve. Pilot: two-tier design - the same app VM (4 vCPU, 16 GB RAM) with the database moved to a managed database service and uploads to object storage. Production target: three-tier design - a containerized API/frontend tier (4 vCPU, 16 GB RAM, autoscaling), a separate AI inference service (8 vCPU or GPU-optional instance, 32 GB RAM) for the NLP credibility model, and a managed database with backups, because the AI workload is the largest and most variable driver identified in Activity 4 and should scale independently of the steady API traffic.

Activity 8: Hypervisor and Virtualization Decision

Approach	Meaning	Best Use
Type 1 hypervisor	Runs directly on physical hardware	Production data center and cloud provider infrastructure
Type 2 hypervisor	Runs above an existing OS	Student lab, local testing, VirtualBox/VMware Workstation
Cloud-managed virtualization	Cloud provider hides hypervisor detail	Most public cloud VM deployments
Container runtime	OS-level process isolation	Lightweight APIs, microservices, fast deployment

Context	Selected Approach	Justification
Local prototype/testing	Type 2 hypervisor (VirtualBox / Docker Desktop)	Lets me build and test the ingestion and scoring pipeline locally without cloud cost before deploying anything
Cloud pilot deployment	Cloud-managed virtualization	Provider manages the hypervisor; I only provision the VM size decided in Activity 7, and can resize quickly during the pilot
Production growth scenario	Cloud-managed virtualization with autoscaling group	Misinformation events cause sudden traffic spikes (Activity 3 peak factor); autoscaling absorbs the surge without manual intervention
Module suitable for container	Container runtime (Docker)	The NLP scoring module and backend API are portable, stateless services that benefit from fast, repeatable CI/CD deployment

Activity 9: Snapshot and Clone Strategy

Snapshot protects a point in time. Clone creates a duplicate environment. Both are useful, but they are not the same.

Event	Snapshot or Clone?	Frequency / Timing	Retention / Reason
Before major deployment	Snapshot	Immediately before every deploy	Keep last 3 snapshots for quick rollback
Before database schema change	Snapshot	Right before running a migration script	Keep until the migration is verified stable (7 days)
Before OS/package update	Snapshot	Just before patching the VM	Keep for 48 hours after the update in case of rollback
Before review/demo	Snapshot	The day before a scheduled review	Keep until the review/demo is completed
Create testing environment	Clone	Whenever a new feature branch needs isolated testing	Delete after the testing cycle ends, to avoid stale duplicate costs
Create team demo copy	Clone	Before a stakeholder or faculty demo	Delete after the demo; it is a working copy, not a backup

Technical Tip: A snapshot is useful for rollback. A clone is useful for parallel testing. A clone should not be treated as the only backup.

Activity 10: VM vs Container Decision

Criteria	Virtual Machine	Container
Isolation	Strong OS-level isolation	Process-level isolation
Startup	Slower	Faster
Resource use	Higher	Lower
Best for	Full OS, legacy app, database VM	Microservices, APIs, portable services
Portability	Moderate	High
Student use	Clear infrastructure learning	Modern deployment learning

Capstone Module	VM / Container / Managed Service	Reason
Frontend	Container	Static/SPA build served fast and portably; redeploys quickly in CI/CD
Backend/API	Container	Stateless service that scales horizontally with traffic
Database	Managed Service	Needs reliable persistence, backups and failover that a managed DB provides out of the box
File storage	Managed Service	Object storage handles article snapshots and reports far more cheaply and durably than VM/container disk
AI/Python module	Container (on a dedicated, larger instance)	Portable for CI/CD, but needs a ring-fenced, larger resource allocation because it is the heaviest workload (Activity 4)
Analytics/dashboard	Container	Lightweight service that scales independently of the core API
Notification/background jobs	Managed Service (serverless functions + queue)	Short-lived, event-triggered tasks fit a pay-per-invocation serverless model best

Activity 11: Final Recommendation

Write the final infrastructure recommendation in 8-12 technical lines. Include VM size, hypervisor/virtualization approach, storage plan, snapshot/clone plan and VM-container decision.

For the CO2 AT1 prototype, NewsTrust runs on a single 4 vCPU / 16 GB RAM / 50 GB disk VM, sized from a CPU calibration score of 10 and a RAM subtotal of ~30 GB, both driven mainly by the NLP credibility-scoring module.

As the pilot moves to real users, the plan splits into a two-tier design: the app/API VM stays at 4 vCPU / 16 GB RAM, while the database migrates to a managed database service so it is not counted against app VM memory.

For production, the design becomes three-tier: a containerized frontend/API tier (4 vCPU, 16 GB RAM, autoscaling group) and a separate AI inference service (8 vCPU or GPU-optional, 32 GB RAM) so the bursty NLP workload scales independently of steady dashboard traffic.

Virtualization moves from a Type 2 hypervisor for local development, to cloud-managed virtualization for the pilot, to cloud-managed virtualization with autoscaling for production, matching the peak-factor traffic spikes expected during viral misinformation events.

Storage totals 50 GB on the VM disk (OS, database index cache, logs) plus a separate, growing object storage bucket (~30 GB and rising) for article snapshots, screenshots and exported fact-check PDFs, which are never kept permanently on VM disk.

Snapshots are taken before every deployment, schema change, OS update and demo, with short, purpose-specific retention windows, while clones are used only for isolated feature testing and demo copies and are never relied on as the sole backup.

The frontend, backend/API, analytics dashboard and AI module are containerized for portability and fast CI/CD, while the database and file storage use managed cloud services for reliability, and background/notification jobs run as serverless functions triggered off a queue.

This plan starts small and transparent for the prototype but does not hide growth: it names the exact point (crossing the 10-point CPU score and the object-storage threshold) at which each component is expected to separate out into its own managed or autoscaled service.

Carry-Forward to CO2 AT2 and CO2 AT3

Output from CO2 AT1	How It Will Be Used Later
VM sizing values	Used as configuration target in CO2 AT2 practical evidence
Hypervisor/virtualization choice	Used to justify deployment environment
Snapshot/clone plan	Used as backup and testing evidence
VM/container comparison	Used in CO2 AT3 infrastructure design case
Final recommendation	Used as the capstone infrastructure baseline

GitHub Submission Checklist

- ☐ Completed workbook file.
- ☐ README with capstone title, sizing summary and final recommendation.
- ☐ Optional architecture diagram or table exported as image/PDF.
- ☐ Repository link submitted in Google Classroom.